

المحاضرة الثامنة: التعامل مع قواعد البيانات SQLite



م. سليمان الشوصي

❖ محاور المحاضرة

- مفهوم قواعد البيانات SQLite.
- تطبيق كامل يعمل لتعامل مع قواعد البيانات بحيث يضم جميع الأوامر الأساسية الخاصة بإنشاء الجداول وإضافة وتعديل وحذف البيانات من قاعدة البيانات SQLite.
- طريقة تمرير البيانات الى صفحة التعديل.
- طريقة بناء الأدوات بكلاسات منعزلة داخل ملف منعزل، من اجل ان يتم استخدامه عدة مرات، ولتقليل كمية الاكواد في الصفحات.
- أدوات إضافية.

• [CircularProgressIndicator](#)

• [ListView.builder](#)

• [showDialog](#)

• [AlertDialog](#) أداة مربع حوار

• [InkWell](#) رز

❖ **ما هو SQLite؟** هو حزمة (Package) شائعة الاستخدام في Flutter تُستخدم لتوفير وصول قاعدة بيانات SQLite لتطبيقات الهواتف المحمولة (Android و iOS).

قاعدة بيانات محلية SQLite: لا يمثل قاعدة بيانات في حد ذاته، بل هو **جسر (Bridge)** يسمح لتطبيق Flutter بالتفاعل مع نظام قاعدة البيانات **SQLite** المضمن في أنظمة التشغيل SQLite. هي قاعدة بيانات **علائقية (Relational)** خفيفة الوزن ومستقلة عن الخادم، حيث يتم تخزين البيانات في **ملف محلي** داخل جهاز المستخدم.

غير متزامن (Asynchronous): جميع عمليات SQLite (مثل الإدخال، والتحديث، والاستعلام) تتم بشكل **غير متزامن** باستخدام مفهوم **Future**، مما يضمن أن عمليات قاعدة البيانات الثقيلة لا تُجمد واجهة المستخدم (UI) للتطبيق.

الاستخدام: يُستخدم لتخزين البيانات التي لا تحتاج إلى مزامنة فورية عبر الإنترنت، مثل: سجلات التطبيق، إعدادات المستخدم، أو البيانات التي يحتاج المستخدم الوصول إليها في وضع عدم الاتصال (Offline Mode).

الخلاصة SQLite: هي الطريقة الأساسية لإدارة قواعد بيانات **SQLite المحلية** في تطبيقات Flutter.

❖ **أولاً: يجب تحميل الحزم التالية من خلال تنفيذ الامر التالية في ال Terminal:**

- عند تنفيذ هذه الأوامر سوف يتم تحميل الحزم بحسب الإصدارات المتوافقة مع إصدارات البيئة المثبتة لدينا.
1. **flutter pub add sqflite**
 2. **flutter pub add sqlite3_flutter_libs**
 3. **flutter pub add sqflite_common_ffi**

❖ أو تحميل المكتبات من خلال متصفح **chrome** باكتابة **flutter pub get sqflite** ونسخها اصدار المكتبة الى الملف **pubspec.yaml** الخاص بتضمين الحزم كما هو موضح في الصورة المقابلة:

```

X home_page.dart • pubspec.yaml • sing_in_page.dart X
pubspec.yaml
14 dependencies:
15   flutter:
16     sdk: flutter
17   cupertino_icons: ^1.0.8
18   sqflite: ^2.4.2
19   sqflite_common_ffi: ^2.3.7+1
20   sqlite3_flutter_libs: ^0.5.42
21
22 dev_dependencies:
23   flutter_test:
24     sdk: flutter
  
```

```

main.dart 1  sql.dart 2  home_page.dart 3  sing_in_page.dart 4  update_page.dart 5  ! a ▶
> sql.dart > MySQL > deletData
1 import 'package:sqflite/sqflite.dart';
2 import 'package:path/path.dart';
3
4 class MySql {
5   static Database? _db;
6   Future<Database?> get db async {
7     if (_db == null) {
8       _db = await intialDb();
9       return _db;
10    } else { return _db; }}
11   intialDb() async {
12     String databasepath = await getDatabasesPath();
13     String path = join(databasepath, 'users.db');
14     Database mydbusers = await openDatabase(path, onCreate: _onCreate,
15       version: 1,onUpgrade: _onUpgrade, );
16     return mydbusers;}
17
18   _onCreate(Database db, int verison) async {
19     await db.execute('''CREATE TABLE "Users" (
20       id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
21       username TEXT NOT NULL,
22       password INTEGER NOT NULL)''');
23     print("تم انشاء قاعدة البيانات و الجدول");
24   }
25   _onUpgrade(Database db, int oldversion, int newversion) async {
26     await db.execute("ALTER TABLE Users ADD COLUMN phone INTEGER ");
27     print("تم تنفيذ امر اضافة عمود الى الجدول السابق");
28   }

```

getDatabasesPath(): دالة تُحدد المسار الافتراضي على الجهاز لتخزين ملفات قواعد البيانات. حيث ان دالة Future تقوم بإرجاع مسار نظام الملفات (String) حيث يجب تخزين ملفات قاعدة بيانات SQLite بشكل آمن على نظام التشغيل) مثل مجلد المستندات في iOS أو مجلد البيانات في Android).

join (): تُنشئ مسار ملف نهائي صحيح ومناسب لنظام التشغيل الحالي. حيث انها دالة من مكتبة path، وظيفتها دمج سلاسل المسارات بشكل صحيح، مع وضع الفواصل المائلة المناسبة (/ أو \) حسب نظام التشغيل (لتجنب أخطاء المسار).

openDatabase(): تُنشئ أو تفتح اتصالاً بقاعدة بيانات SQLite محددة. حيث ان دالة Future تقوم بفتح ملف قاعدة البيانات في المسار المحدد. إذا كان الملف غير موجود، يتم إنشاؤه. تستقبل الدالة دوال رد اتصال مثل onCreate لتنفيذ الإعدادات الأولية للجدول.

خاصية version: تُحدد رقم إصدار قاعدة البيانات.

يُستخدم لمراقبة التغييرات في مخطط (Schema) قاعدة البيانات. إذا كان الإصدار المُعطى في openDatabase أعلى من الإصدار الحالي لقاعدة البيانات المخزنة، يتم استدعاء دالة onUpgrade لتنفيذ تحديثات المخطط.

execute(sql): أوامر تعريف البيانات (DDL) مثل CREATE TABLE و DROP TABLE. حيث انها تقوم بتنفيذ استعلام SQL لا يهدف إلى إرجاع بيانات. يتم استخدامها لـ إنشاء الهيكل (Schema) أو تعديله. لا ترجع قيمة (ترجع Future<void>).

```

readData(String sql) async {
  Database? mydata = await db;
  Future<List<Map>> response = mydata!.rawQuery(sql);
  // "SELECT * FROM Users", ..
};
return response;
}

insertData(String sql) async {
  Database? mydata = await db;
  int response = await mydata!.rawInsert(sql);
  return response;
}

updateData(String sql) async {
  Database? mydata = await db;
  int response = await mydata!.rawUpdate(sql);
  return response;
}

deleteData(String sql) async {
  Database? mydata = await db;
  int response = await mydata!.rawDelete(sql);
  return response;
}

deleteDatabase() async {
  String databasepath = await getDatabasesPath();
  String path = join(databasepath, 'users.db');
  await deleteDatabase(path);
}
}

```

❖ **rawQuery(sql)**: استعلامات اختيار البيانات (DQL) مثل SELECT. حيث تُنفذ استعلام SELECT وترجع قائمة من الخرائط (**List<Map>**) ، حيث يمثل كل خريطة صفًا واحدًا من نتائج الاستعلام.

❖ **rawInsert(sql)**: استعلامات إدراج البيانات (DML) مثل INSERT. دالة تُنفذ استعلام INSERT وترجع رقم صحيح (**int**) يمثل id الصف الجديد الذي تم إدراجه.

❖ **rawUpdate(sql)**: استعلامات تحديث البيانات (DML) مثل UPDATE. حيث تُنفذ استعلام UPDATE وترجع رقم صحيح (**int**) يمثل عدد الصفوف التي تأثرت وتغيرت.

❖ **rawDelete(sql)**: استعلامات حذف البيانات (DML) مثل DELETE. تُنفذ استعلام DELETE وترجع رقم صحيح (**int**) يمثل عدد الصفوف التي تم حذفها.

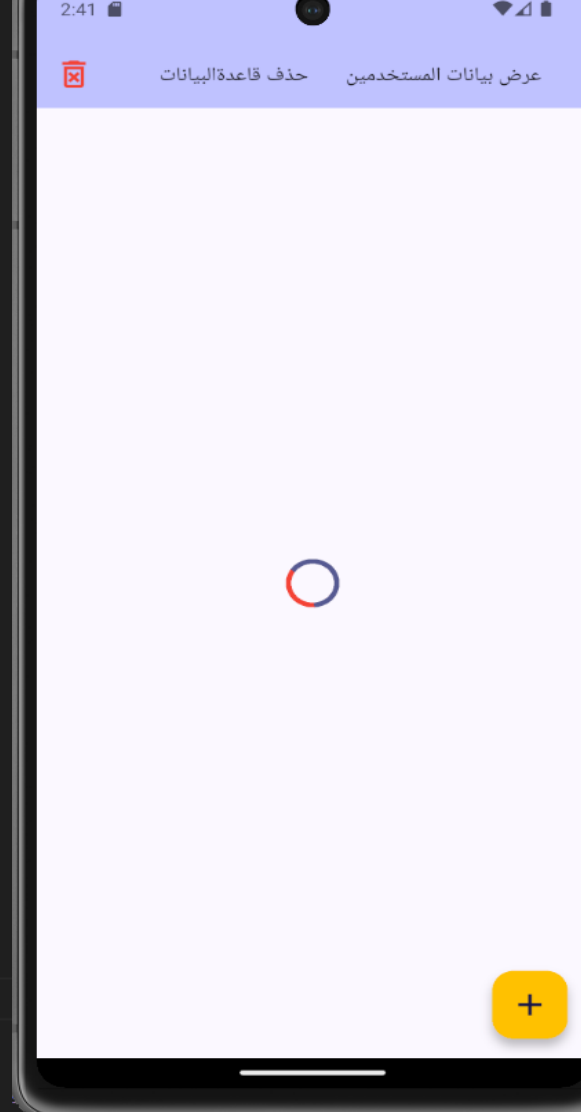
❖ **deleteDatabase()**: تقوم بـ **حذف** ملف قاعدة البيانات كاملاً من نظام الملفات. حيث انها تستقبل المسار الكامل للملف وتقوم بحذفه نهائياً من الجهاز، مما يزيل جميع البيانات والهيكل المخزن

```
import 'package:flutter/material.dart';
import 'package:lesson_1/sqlflite/home_page.dart';
import 'package:lesson_1/sqlflite/sing_in_page.dart';
import 'package:lesson_1/sqlflite/update_page.dart';
// import 'package:sqlflite_common_ffi/sqlflite_ffi.dart';
Run | Debug | Profile
void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const MyApp());
}
class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(title: 'Flutter Demo',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorScheme: ColorScheme.fromSeed(
        seedColor: const Color.fromARGB(255, 18, 22, 252),
      )), // ColorScheme.fromSeed // ThemeData
      home: MyHomePage(), // SingInPage(),
      routes: {
        "adduserspage": (context) => SingInPage(),
        "myHomepage": (context) => MyHomePage(),
        "UpdateDataPage": (context) => UpdateDataPage(),
      }
    );
  }
} // MaterialApp
```

```

2 import 'package:flutter/material.dart';
3 import 'package:lessons_sqlflite/sql.dart';
4 import 'package:lessons_sqlflite/update_page.dart';
5 class MyHomePage extends StatefulWidget {
6   const MyHomePage({super.key});
7   @override
8   State<MyHomePage> createState() => _MyHomePageState();
9 }
10 class _MyHomePageState extends State<MyHomePage> {
11   MySql sql = MySql();
12   bool isloading = true;
13   List dataUser = [];
14   Future readData() async {
15     List<Map> response = await sql.readData("SELECT * FROM Users");
16     dataUser.addAll(response);
17     isloading = false;
18     if (this.mounted) { setState(() {}); }
19     print(dataUser);
20   }
21   @override
22   void initState() { readData(); super.initState(); }
23   @override
24   Widget build(BuildContext context) {
25     return Scaffold(appBar: AppBar(leading: IconButton(color: Colors.red,
26       icon: Icon(Icons.delete_forever_outlined, size: 30),
27       onPressed: () async { sql.deleteDatabase(); }, // IconButton
28       backgroundColor: Theme.of(context).colorScheme.inversePrimary,
29       title: Center(
30         child: Text('عرض بيانات المستخدمين', style: TextStyle(color: Colors.red, fontStyle: FontStyle.italic),), // Center // AppBar
31       ), // Center // AppBar
32       floatingActionButton: FloatingActionButton(
33         backgroundColor: Colors.amber, // كود زر الانتقال الى صفحة الاضافة
34         onPressed: () { Navigator.of(context).pushNamed("adduserspage"); },
35         child: Icon(Icons.add, size: 30), // FloatingActionButton

```



الأداة CircularProgressIndicator: هي (Widget) تستخدم لعرض مؤشر بصري على أن هناك عملية جارية تتطلب وقتاً لإكمالها (مثل جلب البيانات، أو معالجة طويلة).

طريقة العمل: تعرض حركة دائرية مستمرة (عادةً ما تكون حلقة تدور وفقاً لنمط Material Design لتمثل حالة الانتظار).

```

37 ? Center(child: CircularProgressIndicator(
38   padding: EdgeInsets.all(20), backgroundColor: Colors.red,)) // CircularProgressIndicator //
39 : Container(width: double.infinity,
40   margin: EdgeInsets.all(5),
41   child: ListView(children: [
42     ListView.builder(itemCount: dataUsers.length,
43       physics: NeverScrollableScrollPhysics(),
44       shrinkWrap: true,
45       itemBuilder: (context, i) {
46         return Card( color: const Color.fromARGB(255, 223, 219, 176),
47           child: Row( children: [
48             Expanded( flex: 3, child:
49               ListTile(title:Text( "name: ${dataUsers[i]['username']}",style: TextStyle(fontSize: 12)),
50                 subtitle: Text("password: ${dataUsers[i]['password']}",style: TextStyle(fontSize: 13),),
51                 leading: Text("id : ${dataUsers[i]['id']}",style: TextStyle(fontSize: 14),
52                 )), // Text // ListTile // Expanded
53             Expanded(flex: 1, child:
54               Column( children: [
55                 mybuttonIcon( // استدعاء كود بناء زر التعديل
56                   iconcolors: Colors.black, myiconsize: 20,
57                   backgroundColors: const Color.fromARGB(255, 182, 174, 243),
58                   iconimg: const Icon(Icons.edit),
59                   onPressedButton: () {
60                     Navigator.of(context).push(MaterialPageRoute(
61                       builder: (context) => UpdateDataPage(// rawDataUsers: dataUsers[i],
62                         name: dataUsers[i]['username'],
63                         password: dataUsers[i]['password'],id: dataUsers[i]['id'],));)), // UpdateDataPage
64                 mybuttonIcon( // استدعاء كود بناء زر الحذف
65                   iconcolors: Colors.red,myiconsize: 25,
66                   backgroundColors: const Color.fromARGB(255, 183, 176, 239), //Colors.red,
67                   iconimg: const Icon(Icons.delete_forever),
68                   onPressedButton: () async {
69                     int delete = await sql.deleteData("DELETE FROM Users WHERE id= ${dataUsers[i]['id']}",);
70                     if (delete > 0) { print("تم حذف حساب المستخدم : ${dataUsers[i]['username']}");
71                     dataUsers.removeWhere((element) =>element['id'] ==dataUsers[i]['id'], );
72                     setState(() {});
73                     // Navigator.of( context).popAndPushNamed("myHomepage");

```



❖ أداة التعامل مع البيانات غير المترامنة والقوائم في Flutter

ListView.builder: هي ودجت مخصصة لإنشاء قوائم طويلة أو لا نهائية بكفاءة عالية. تستخدم لبناء العناصر عند الحاجة إليها فقط (lazily)، أي يتم بناء العناصر التي تظهر على الشاشة فقط، مما يوفر في الذاكرة وأداء التطبيق. طريقة عملها:

- تتطلب دالة **itemBuilder** التي تحدد كيف يتم بناء كل عنصر.
 - تتطلب خاصية **itemCount** لتحديد عدد العناصر الإجمالي.
- الاستفادة: مثالية لعرض قوائم ضخمة من المنتجات، الرسائل، أو النتائج المسترجعة من قاعدة البيانات
- ومن أهم الخصائص المستخدمة وهي التالي:

- **physics: NeverScrollableScrollPhysics()**: هي خاصية تتحكم في فيزياء (سلوك) التمرير للودجت القابلة للتمرير (مثل ListView). بتعيينها إلى **NeverScrollableScrollPhysics**، يتم إلغاء خاصية التمرير للقائمة بالكامل.
- الاستفادة والاستخدام الشائع: تُستخدم بكثرة عند وضع قائمة (ListView) داخل قائمة أخرى قابلة للتمرير (مثل SingleChildScrollView) لتجنب تعارض التمرير (Nested Scrolling). إذا لم يتم إلغاء التمرير للقائمة الداخلية، فإنها ستتنافس مع الودجت الأب القابلة للتمرير على معالجة إيماءة التمرير.
- **shrinkWrap: true**: هي خاصية منطقية (Boolean) تُستخدم في الودجت القابلة للتمرير مثل ListView. عند تعيينها إلى **true**، فإنها تجعل القائمة تأخذ أقل مساحة عمودية (أو أفقية) ممكنة بناءً على محتوى عناصرها. أي أن ارتفاعها سيكون فقط بقدرة ارتفاع العناصر الموجودة بداخلها.
- طريقة العمل (الأهمية): بشكل افتراضي، تحاول ListView أن تأخذ المساحة المتاحة بالكامل. لكن عند استخدامها داخل ودجت أخرى ذات ارتفاع غير محدود (مثل Column أو SingleChildScrollView)، يجب استخدام **shrinkWrap: true** لحل قيود التخطيط، مما يسمح لها بالنمو بناءً على محتواها.
- الاستفادة: حل مشكلة "الارتفاع غير المحدود" (Unbounded Height) عند دمجها مع ودجت تسمح بالتمرير الخارجي.
- **itemBuilder: (context, i)**: هي دالة معاودة الاتصال (Callback Function) إلزامية لـ **ListView.builder**. مهمتها هي بناء الودجت الخاصة بعنصر واحد في القائمة. المُدخلات:
- **context**: السياق (BuildContext) الذي يتم فيه بناء الودجت.
- **i**: المؤشر (Index) الحالي للعنصر الذي يتم بناؤه في القائمة (يبدأ من صفر).

❖ بنية كود كلاس ازرار التعديل والحذف :

```

buttonicon.dart •
lib > sqflite > widgets > buttonicon.dart > ...
1 import 'package:flutter/material.dart';
2 class mybuttonIcon extends StatelessWidget {
3   // تعريف المتغيرات الخصائص الزر
4   final Color iconcolors;
5   final Color bacgroundcolors;
6   final Icon iconimg;
7   final Function onPressedButton;
8   final double myiconsize;
9
10  //constractor البينات عن طريق دالة الباني
11  const mybuttonIcon({
12    super.key,
13    required this.iconcolors,
14    required this.bacgroundcolors,
15    required this.iconimg,
16    required this.onPressedButton,
17    required this.myiconsize, });
18  @override
19  Widget build(BuildContext context) {
20    ⚡ return Card(color: bacgroundcolors,
21      child: IconButton(
22        onPressed: () => onPressedButton(),
23        icon: iconimg,
24        color: iconcolors,
25        iconSize: myiconsize,
26        )); } // IconButton // Card

```

❖ كود بنية كلاس صندوق تعبئة البيانات (الحقول الادخال والتعديل) :

```

1 import 'package:flutter/material.dart';
2
3 class TextEditFiledData extends StatelessWidget {
4   final TextEditingController textcontroller;
5   final String text;
6   final TextInputType kepordType;
7   const TextEditFiledData({
8     super.key,
9     required this.text,
10    required this.textcontroller,
11    required this.kepordType,
12  });
13  @override
14  Widget build(BuildContext context) {
15    return TextField(
16      controller: textcontroller,
17      keyboardType: kepordType,
18      decoration: InputDecoration(
19        filled: true,
20        border: OutlineInputBorder(
21          borderRadius: BorderRadius.circular(10)),
22        labelText: text,
23        labelStyle: const TextStyle(fontSize: 16,
24          fontWeight: FontWeight.bold), //
25        ), ); } // InputDecoration // TextField
26

```

```

2 import 'package:flutter/material.dart';
3 import 'package:lesson_1/sql.dart';
4 import 'package:lesson_1/sqlflite/widgets/textfromfile.dart';
5 import 'widgets/showDialog_function.dart';
6 class SingInPage extends StatefulWidget {
7   SingInPage({Key? key}) : super(key: key);
8   @override
9   SingInPageState createState() => _SingInPageState();
10 class _SingInPageState extends State<SingInPage> {
11   MySql sqldb = MySql();
12   TextEditingController userName=TextEditingController();
13   TextEditingController password=TextEditingController();
14   @override
15   Widget build(BuildContext context) {
16     return Scaffold(
17 > appBar: AppBar( ...
18 > title: Center( // Center // AppBar ...
19 body: Container(padding: EdgeInsets.all(15),
20   child: ListView( children: [
21     Icon(size: 70, Icons.person),
22     SizedBox(height: 15),
23     // استدعاء كلاس صندوق المدخلات الحقول
24     TextEditFiledData(text: "ادخل اسم المستخدم",
25       textcontroller: userName,
26       keporType: TextInputType.text,), // TextEditFiled
27     SizedBox(height: 15),
28     TextEditFiledData( text: "كلمة السر",
29       textcontroller: password,
30       keporType: TextInputType.number,), // TextEditFil
31     SizedBox(height: 15),
32     InkWell( child: SizedBox( width: 100,

```

```

InkWell( child: SizedBox( width: 100,
  child: Card(color: const Color.fromARGB(255,247,196,45),
    child: Padding( padding: EdgeInsets.all(5),
      child: Center(child: Text("اضافة مستخدم")),)), // Padd
  onTap:() async{ // شرط التحقق من ان الصناديق ليست فارغة
    if (userName.text != "" && password.text != "") {
      int response = await sqldb.insertData(
        '''INSERT INTO Users ("username", "password")
        VALUES("${userName.text}", "${password.text}")''',);
      // int response = await sqldb.insertUser("Users", {...
      print(response);
      // print("username= ${userName.text}"); ...
      Navigator.of(context).popAndPushNamed("myHomepage");
    } else {
      // عندما تكون الحقول فارغة AlertDialog استدعاء الدالة عرض اداة
      showCustomErrorDialog(context);
    }
  }},
)] ,),,);}} // InkWell // ListView // Container // Scaffold

```

❖ واجهة الإضافة
الى قاعدة
البيانات.

```

1 import 'package:flutter/material.dart';
2 void showCustomErrorDialog(BuildContext context) {
3   showDialog(
4     context: context, // سياق الشاشة الحالية
5     builder: (BuildContext dialogContext) {
6       return AlertDialog( titleTextStyle: TextStyle(
7         color: Colors.red,
8         fontWeight: FontWeight.bold, // TextStyle
9         contentTextStyle: TextStyle(color: Colors.black,
10          fontWeight: FontWeight.bold,
11          ), // TextStyle
12          title: const Text('!خطأ في الإدخال'),
13          content: const Text(
14            "لم يتم إضافة البيانات، يُرجى التأكد من الأخطاء في الحقول",
15            ), // Text
16            actions: <Widget>[
17              TextButton( onPressed: () {
18                // Dialog يغلق الى Navigator.pop(context). إغلاق مربع الحوار
19                Navigator.pop(dialogContext);
20              },
21              child: Card(color: Colors.amber,
22                child: Container( padding: EdgeInsets.all(10),
23                  child: const Text('حسنأ',
24                    style: TextStyle(fontWeight: FontWeight.bold),
25                  ), // Text
26                ), // Container
27              ), // Card // TextButton
28            ]); // <Widget>[] // AlertDialog

```

❖ أداة AlertDialog مربع حوار مخصص

تُستخدم الدالة showDialog مع الودجت AlertDialog لعرض واجهات متراكبة مؤقتة (Overlay Routes) فوق الشاشة الحالية لتنبيه المستخدم أو التفاعل معه.

١. آلية عمل الدالة (showDialog)

طبيعتها: دالة تنفيذية (Helper Function) تقوم بدفع (Push) مربع الحوار كمسار جديد في قمة مكس التنتقل (Navigator Stack).
السياق البرمجي (context): يُحدّد موقع استدعاء النافذة ضمن شجرة الودجت (Widget Tree) لتمكين الـ Navigator من إدارتها.

دالة البناء (builder): دالة معاودة اتصال (Callback) تُرجع هيكل النافذة، وتزودها بسياق منفصل خاص بها (dialogContext).

٢. مكون الواجهة (AlertDialog)

طبيعتها: وديجت جاهزة تعتمد نمط Material Design لتأطير وبناء العناصر البصرية للحوار.
الهيكل الأساسي:

- **العنوان (title):** الهيدر الرئيسي لتوضيح طبيعة التنبيه.
- **المحتوى (content):** جسم النافذة لعرض النصوص أو العناصر التفاعلية.
- **الإجراءات (actions):** الأزرار المخصصة لاتخاذ القرار (مثل: موافق / إلغاء).



```

import 'package:flutter/material.dart';
import 'package:lessons_sqlite/home_page.dart';
import 'package:lessons_sqlite/sql.dart';
import 'package:lessons_sqlite/widgets/textfromfiled.dart';
class UpdateDataPage extends StatefulWidget {
  final name;
  final password;
  final id;
  // final Map rawDataUsers;
  UpdateDataPage({ Key? key, this.name, this.password, this.id,
    // required this.rawDataUsers,
  }) : super(key: key);
  @override
  _UpdateDataPageState createState() => _UpdateDataPageState();
}
class _UpdateDataPageState extends State<UpdateDataPage> {
  MySql sqldb = MySql();
  TextEditingController userName = TextEditingController();
  TextEditingController password = TextEditingController();
  @override
  void initState() {
    // userName.text = widget.rawDataUsers['username'].toString();
    // password.text = widget.rawDataUsers['password'].toString();
    userName.text = widget.name.toString();
    password.text = widget.password.toString();
    super.initState();
  }
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(backgroundColor: Theme.of(context).colorScheme.inversePr
        title: Center(child: Text( "مفحة تعديل بيانات المستخدم", // Text //
47 body: Container(padding: EdgeInsets.all(15),
48 child: ListView( children: [ Icon(size: 70, Icons.person),
49   SizedBox(height: 15),
50   TextEditFiledData( text: "تعديل اسم المستخدم",
51   textcontroller: userName, keporType: TextInputType.text, ), // Te
52   SizedBox(height: 15),
53   TextEditFiledData( text: "تعديل كلمة السر",
54   textcontroller: password,
55   keporType: TextInputType.number, ), // TextEditFiledData
56   SizedBox(height: 15),
57   InkWell(child: SizedBox(width: 100,
58   child: Card(color: const Color.fromARGB(255, 247, 196, 45),
59   child: Padding( padding: EdgeInsets.all(5),
60   child: Center(child: Text("تعديل البيانات")),)), // Padd
61   onTap: () async {
62   int response = await sqldb.updateData(
63   ''' UPDATE Users SET username = "${userName.text}",
64   password = "${password.text}" WHERE id = ${widget.id}''',);
65   // int response = await sqldb.updateUser("Users", {...
66
67   if (response > 0) {
68   // ترجع الى الصفحة السابقة بدون عمل تحديث لمحتوى الصفحة
69   // Navigator.of(context).pop();
70   // MaterialPageRoute(builder: (context) => MyHomePage())
71   print("===== $response");
72   Navigator.of(context).pushAndRemoveUntil(
73   MaterialPageRoute(builder: (context) => MyHomePage()),
74   (Route<dynamic> route) => false, );}
75   ])),);} // InkWell // ListView // Container // Scaffold
76   }
77
78
79

```



❖ المطلوب تنفيذ محتوى المحاضرة ويتم مناقشته في المحاضرة القادمة.

Thank you